A metaheuristic–exact hybrid for the *complete* efficient set of a multi-objective integer program, with a linear fractional decision criterion

Abstract

Let $D = \{x \in \mathbb{Z}^n_+ : Ax \leqslant b\}$ be bounded and let $Z_1, \ldots, Z_p$ be criteria to maximise, each a ratio of affine forms with a strictly positive denominator on D. We address two questions about the efficient set $E(P_D)$: producing *all* of it, and maximising a further linear fractional criterion Φ over it.

Existing methods for the second question work in *decision* space, truncating D with Sylva–Crema cuts. Each cut is a disjunction and costs p binary variables and $p+1$ big-M rows, so every sub-problem is strictly larger than the last. We keep the unexplored part of the problem in *criterion* space instead, as a list of boxes: truncation becomes a box subtraction, every sub-problem is D plus a handful of ordinary linear rows, and the model never grows.

Four results follow. First, a characterisation that makes the complete efficient set cheap: **if v is non-dominated then every $x \in D$ with $Z(x) = v$ is efficient**, so $E(P_D)$ is exactly the union of the slices of the non-dominated set and *no efficiency test is paid on any slice point*. Second, a slice is D plus p *linear* equations, for fractional criteria as much as for linear ones; solving those equations eliminates r variables before enumeration begins and is worth a median 49× over scanning with them as pruning rows. Third, a hybridisation that pays: a Pareto local search supplies verified efficient points which are placed in the box list without a probe. Fourth, and it is what the first three are for: an explicit $E(P_D)$ admits *any computable* decision function, where the decision-space methods need a ratio form to build their cuts from. A sum of ratios, a maximum of ratios, or several decision functions at once — each its own problem in that literature — cost here exactly what one ratio costs. The break-even against solving one decision function at a time is a measured median of 2.4 functions over 117 instances, counted in sub-programs so that it is deterministic; for a single linear fractional Φ the decision-space method is often the better buy, and the break-even is 1 only outside the class it accepts.

The hybrid reaches the whole of $E(P_D)$ at 1.36× the pure exact method over 120 instances spanning $n = 6 \ldots 12$, faster on 116, at 1.39× on genuinely fractional criteria, and between 1.14× and 1.37× on each of five structurally different instance families, with identical answers throughout.

The hybridisation only pays when the seeds are handed over in the order the exact search's own probe would meet them; in the order the archive produces them the same hybrid is *slower*. We also report, at the same length, four changes that remove a quarter of the sub-problems and buy nothing, and a single-program completeness certificate that is sound and 4.5× slower, because the negative results delimit where hybridisation can help at all.

1 Problem and notation

$$(P_D) \qquad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \ldots, p, \qquad x \in D, \tag{1}$$

with $D = \{x \in \mathbb{Z}^n_+ : Ax \leqslant b\}$ non-empty and bounded and every denominator $D_k(x) = d_k^\top x + b_k$ strictly positive on D. **Both assumptions are enforced**, because violating either fails silently rather than loudly. An unbounded probe returns a status that is not *optimal*, which the enumeration read as “this box is empty”: on $D = \{x \in \mathbb{Z}^2_+ : x_1 - x_2 \leqslant 0\}$, feasible and

with efficient points, it returned an *empty front marked complete*, and both optimisation searches reported *infeasible*. Since $x \geqslant 0$, boundedness is one integer program for the whole problem.

The positive-denominator assumption is load-bearing twice over: the form (3) carries the sign of $Z_k(x) - Z_k(\bar{x})$ only because both denominators are positive, and the big-M of (4) is a valid bound only under the same condition. A criterion violating it does not fail loudly — on $Z_1 = x_1/(x_1 - 1)$ over $0 \leqslant x \leqslant 3$ the enumeration returns a single vector and reports it *complete* while a feasible point dominates it — so the entry points verify it in one integer program per genuinely fractional criterion, and none at all for linear ones. A point $x \in D$ is *efficient* when no $y \in D$ satisfies $Z(y) \geqslant Z(x)$ with at least one strict inequality; $E(P_D)$ denotes the set of efficient points and $F = Z(E(P_D))$ the non-dominated set. Linear criteria are the degenerate case $d_k = 0$, $b_k = 1$.

Given a decision maker's criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$,

$$(P_E) \qquad \max \ \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \tag{2}$$

$E(P_D)$ is a union of faces of D: non-convex, with no explicit description.

Throughout, the data of each criterion is scaled to integers (numerator and denominator by the same factor, leaving the ratio untouched). That makes the quantity

$$e_k(x; \bar{x}) \ = \ D_k(\bar{x})\, D_k(x) \left(Z_k(x) - Z_k(\bar{x})\right) \tag{3}$$

linear in x and integer-valued, so that $Z_k(x) > Z_k(\bar{x}) \iff e_k(x; \bar{x}) \geqslant 1$ exactly — no minimal step has to be estimated, which is what makes the transposition from linear to fractional criteria faithful rather than approximate.

Remark 1. *Φ is a function of x and **not** of $Z(x)$. Two efficient points with the same criterion vector can have different Φ. Every criterion-space construction below has to account for this, and it is the reason the transfer of criterion-space search from linear to fractional objectives is not immediate.*

2 Where the cost of the decision-space method comes from

One iteration of the published approach maximises Φ over a truncated region D_l, tests the maximiser for efficiency, banks the best Φ on the criterion slice about to be destroyed, and truncates: delete $\{x : Z(x) \leqslant Z(\tilde{x})\}$.

Keeping the complement means keeping the x for which *some* criterion strictly improves on $\tilde{x}$. That is a disjunction, not a system of linear inequalities, and it is modelled with one binary per criterion:

$$e_k(x; \tilde{x}) \geqslant 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \ldots, p, \qquad \sum_{k=1}^{p} y_k \geqslant 1, \tag{4}$$

with M_k a lower bound on e_k over the region. **Every cut therefore adds p binaries and $p+1$ rows**, and the late sub-problems cost far more than the early ones. This fixes the cost model used throughout:

$$\text{total cost} \ \approx \ (\text{number of sub-problems}) \times (\text{cost of one, which grows}).$$

3 The same truncation in criterion space

Keep the unexplored region as a list of *boxes* instead. Deleting $\{Z \leqslant Z(a)\}$ from a box is a box subtraction, and it has a disjoint p-way decomposition:

$$\text{child } k: \quad e_k(x; a) \geqslant 1 \ \text{ and } \ e_j(x; a) \leqslant 0 \ \text{ for } j < k. \tag{5}$$

Child k asks that Z_k strictly improve on a while no earlier criterion does. The children are disjoint, they cover the box minus $\{Z \leqslant Z(a)\}$, and — this is the point — each is D plus a handful of ordinary linear rows. **The model never grows.** A box is written with the integer-valued rows (3) rather than with numeric bounds on Z_k, which is what carries the construction to fractional criteria unchanged.

Each box also carries, for nothing, an interval per criterion: child k raises lo_k to $Z_k(a)$ and lowers hi_j to $Z_j(a)$ for $j < k$. We use it in Section 9.

4 The complete efficient set

4.1 Enumerating the non-dominated set

The enumeration keeps a list of boxes covering the unexplored region. It pops a box, *probes* it by maximising a fixed strictly positive direction over D intersected with the box, repairs the maximiser to an efficient point a, records $Z(a)$, and splits the box around a by (5).

Proposition 1. *The enumeration terminates and records every non-dominated vector exactly once.*

Proof. Termination. The probe returns a point x of D inside the box, and the efficient point a it is repaired to dominates it, so $Z(x) \leqslant Z(a)$ and the split around a removes at least $Z(x)$ from that box. The unexplored region shrinks on every pass, and the non-dominated set of a bounded integer program is finite.

Completeness. A non-dominated v leaves the region only through a split around some centre a with $v \leqslant Z(a)$. If $v \neq Z(a)$ that says a dominates v, which no non-dominated vector admits. So v leaves only at the moment it is recorded; the loop ends only when the region is empty, so every v is recorded, exactly once since centres are de-duplicated by criterion vector. □

4.2 From the non-dominated set to the efficient set

Completeness of F is not completeness of $E(P_D)$: one vector can be attained by several efficient points and the enumeration keeps one per vector. The gap closes on one observation.

Theorem 1. *If v is non-dominated, then **every** $x \in D$ with $Z(x) = v$ is efficient.*

Proof. Suppose some $y \in D$ dominated such an x. Then $Z(y) \geqslant Z(x) = v$ with at least one strict inequality, which says v is dominated — against the hypothesis. □

Corollary 1. $E(P_D) \;=\; \bigcup_{v \in F} \{\, x \in D : Z(x) = v \,\}$, *and **no efficiency test is paid on any slice point**.*

The dearest operation in the whole method — the efficiency test — is simply not invoked on the slices. What remains is to enumerate each slice.

4.3 A slice is D plus p linear equations

Fixing $Z(x) = v$ is linear in x *even for fractional criteria*: since $D_k(x) > 0$ on D, the equation $(c_k^\top x + a_k)/(d_k^\top x + b_k) = v_k$ clears to

$$(c_k - v_k d_k)^\top x \;=\; v_k b_k - a_k, \tag{6}$$

with no sign condition and no case split. A slice therefore carries no binary, no big-M and nothing that grows from one slice to the next.

These are equations, so *solve* them rather than use them to prune. Gaussian elimination in exact rationals expresses r of the variables as linear functions of the rest; substituting into every

row of the model leaves an equivalent system in $n - r$ unknowns, on which the enumeration runs. Two rows per eliminated variable keep it inside its own bounds. At each leaf the eliminated variables are rebuilt and the one condition that is *not* linear — that they come out integer — is checked there. An inconsistent system settles the slice as empty with no enumeration at all. The search space $(ub + 1)^n$ becomes $(ub + 1)^{n-r}$.

Table 1: Pruning on the slice equations against solving them. Fourteen instances, $n = 7 \dots 12$; identical point sets throughout.

instance	prune	eliminate	ratio
$n = 9$, $p = 3$	29.71 s	0.31 s	96.7×
$n = 10$, $p = 5$	119.47 s	0.92 s	129.9×
$n = 12$, $p = 3$	796.84 s	8.34 s	95.5×
median of all fourteen			**49.2×**

Every coefficient is a rational and the elimination is exact, so this changes the order of the enumeration and nothing about its output.

4.4 Pairing each vector with a maximiser of Φ

By Remark 1 the front alone does not answer (P_E). Each vector is therefore paired with the maximiser of Φ *on its slice*, at one extra integer program per vector. That pairing has a consequence worth stating: **the largest value on the front is the optimum of** (P_E), so the enumeration is a second, independent exact method for (P_E) and serves as a cross-check on the first.

5 What an explicit $E(P_D)$ buys: the admissible decision function

The previous section delivers $E(P_D)$ itself rather than an optimum, and that changes what Φ has to be. The point is easiest to see as a contrast between what each route needs:

> A decision-space method needs *structure in* Φ and nothing of $E(P_D)$; this route needs $E(P_D)$ and nothing of Φ.

A branch-and-cut for a linear fractional Φ derives its cuts, its bounds and its Dinkelbach-style sub-problems from the ratio form. Replace Φ by a *sum* of two ratios and the method does not degrade — it stops applying, because there is no ratio to linearise. Over a finite, explicit, certified $E(P_D)$, that case costs exactly what one ratio costs, since the optimum of a function over a finite explicit set is settled by evaluation and not by structure.

Theorem 2 (Admissible class)**.** *Let $E(P_D)$ be known and proved complete. Then for* any *function f defined on D,*

$$\max\{f(x) : x \in E(P_D)\}$$

is computed exactly by evaluating f on $E(P_D)$, together with the full set of maximisers. No linearity, monotonicity, continuity, differentiability or ratio form is required of f.

The proof is the finiteness of $E(P_D)$ and nothing else, which is exactly why the statement is worth making: the content is not in the theorem but in having paid for $E(P_D)$ in Section 4 at a price that makes paying worthwhile. Table 2 reports six shapes of f checked against independent brute-force maximisation; the first is the classical case and the other five are outside the class any of the decision-space fractional methods accepts.

Table 2: Six decision functions on the published instance, each exact against a brute-force maximisation over the reference efficient set. Φ_1, Φ_2 are linear fractional.

decision function	admitted by decision-space branch-and-cut?	optimum
Φ_1	yes — the classical case	5/17
$\Phi_1 + \Phi_2$	no	47/35
$\max(\Phi_1, \Phi_2)$	no	6/5
$\Phi_1 \cdot \Phi_2$	no	6/35
$(x_1^2 + 1)/(x_2^2 + 1)$	no	10
piecewise / black box	no	3/11

5.1 Several decision functions at once

The same explicitness answers a harder question for free. Optimising *two* linear fractional functions over an integer efficient set is its own problem in the literature, needing its own branch and bound with cutting planes. Here it is a dominance filter over a set already in hand:

$$\left\{ x \in E(P_D) \;:\; \text{no } y \in E(P_D) \text{ has } (\Phi_1, \ldots, \Phi_m)(y) \geqslant (\Phi_1, \ldots, \Phi_m)(x) \text{ with one strict} \right\}$$

and the cost does not depend on m, nor on what the Φ_i are: m ratios, one ratio and one quadratic, or ten black boxes cost the same scan. On the published instance the two-ratio filter keeps 4 of the 7 efficient points, exactly the brute-force answer.

But asking more is not the same as learning more, and the limit is measured. As m grows the non-dominated subset grows towards the whole of $E(P_D)$, and a filter that returns everything has told the decision maker nothing.

Table 3: Share of $E(P_D)$ surviving the filter, by number of decision functions and by $|E(P_D)|$. Thirty instances per configuration, pooled by size; † marks a row resting on fewer than fifteen instances. Generated from the campaign's JSON.

$\lvert E(P_D)\rvert$	inst.	m=1	m=2	m=3	m=4	m=5	m=6	m=8	saturates
5	54	25%	37%	69%	100%	100%	100%	100%	$\mathbf{m = 4}$
9	41	11%	25%	46%	73%	85%	100%	100%	$\mathbf{m = 6}$
18†	8	6%	27%	40%	44%	59%	97%	100%	$\mathbf{m = 8}$
28†	4	4%	11%	30%	30%	69%	87%	100%	$\mathbf{m = 8}$

So the capability is free and the *answer* is not. The saturation point rises with $|E(P_D)|$ — $m = 4, 6, 8, 8$ for efficient sets of about $5, 9, 18$ and 28 points — but only slowly, and past it a decision maker who asks another question gets the whole efficient set back. The growth is consistent with being logarithmic in $|E(P_D)|$, which is what the classical result on the expected size of a non-dominated subset would suggest, but four buckets on a coarse grid in m cannot establish that and it is not claimed here.

Ties are reported rather than resolved arbitrarily, which the single-answer interface of the decision-space methods cannot do and which matters here for the reason in Remark 1: Φ is a function of x and not of $Z(x)$, so several efficient points may attain the optimum while carrying different criterion vectors.

5.2 What it costs, and when it is the wrong choice

This is a trade and not a free lunch, so the break-even is measured rather than asserted. Against the honest alternative — running the exact method of Section 6.4 once per decision function — building $E(P_D)$ first pays only after enough questions are asked of it.

Table 4: How many decision functions before building $E(P_D)$ pays, in **sub-programs**: (programs to build $E(P_D)$)/(programs for one exact run), a deterministic quantity since a scan solves none. Thirty instances per configuration; generated from the campaign's JSON by `studies/tables.py`, not transcribed.

configuration	inst.	cens.	$\lvert E(P_D)\rvert$	break-even, sub-programs
$n = 5$, $p = 3$	30	0	4	**1.8** [1.6, 2.5]
$n = 6$, $p = 3$	30	0	6	**2.2** [2.0, 2.5]
$n = 6$, $p = 4$	27	3	9	**3.1** [2.1, 4.2]
$n = 7$, $p = 3$	30	0	8	**2.6** [2.0, 3.3]
all	117			**2.4** [1.8, 2.9]

> **This table is not the comparison between the two methods.** It prices one deliverable against a different one — building the whole of $E(P_D)$ against answering a single question — and the first is of course dearer. The like-for-like comparison, both methods answering the same (P_E), is Table 16: criterion space wins every row, from 1.52× at $p = 2$ to 14.57× at $p = 6$, with the censoring one-sided in the other method's favour. What follows is the price of the *extra* product, not a deficit on the shared one.

The pooled median is **2.4** decision functions, with an interquartile range of [1.8, 2.9] over 117 instances and extremes of 1.2 and 7.2.

The statistic is in sub-programs and not in seconds, which is why it can be quoted this tightly. A scan over an explicit $E(P_D)$ solves *no* integer program at all, so the break-even in programs is exactly (programs to build $E(P_D)$)/(programs for one exact run) — a deterministic quantity, identical on every machine and every repeat, immune to the ±30% noise floor that no number of repeats removes. The seconds in the campaign's own output are the noisy version of the same number and sit below 1.

An earlier version of this section reported a median of 13 with a range of 1 to 2117. That is withdrawn. It came from eight hand-made instances with a deliberately conflicting criterion structure, where $\lvert E(P_D)\rvert$ ran from 17 to 105; the campaign family has $\lvert E(P_D)\rvert$ between 4 and 9, and building a small efficient set is cheap. The old figure was not a wrong measurement — it is what those eight instances did — but it was quoted as the break-even when it was the break-even *of one family*, and generalising from a single family is the error this work records committing five times before. This is the sixth, caught by re-measuring at the campaign's standard.

Three of the 120 runs were censored, all at $p = 4$, when an exact run spent its budget. That censoring is one-sided *against* the method argued for here: an exact run that overruns is an expensive one, and an expensive exact run is precisely the case where building $E(P_D)$ pays soonest, so dropping it removes the instances most favourable to the scan. The true break-even is therefore below 2.4, not above it.

Two consequences, and the first is the unflattering one. **For a single linear fractional Φ, the decision-space method is often the better buy**, and nothing here contradicts that: it never builds $E(P_D)$ and so is untouched by its size, while this route builds $E(P_D)$ and is bounded by it. The two fail on different instances — on a structureless f, and on an $E(P_D)$ too large to enumerate — which is why they belong side by side rather than ranked.

The second does not depend on the measurement at all. For any f outside the linear fractional class, the break-even is 1, because there is no competitor to amortise against.

6 The hybridisation

6.1 What a seed can feed here, and what it cannot

A seed for an *optimisation* search feeds one mechanism: the bound. A search that already knows a good value prunes boxes whose bound cannot beat it. **The enumeration has no bound and no incumbent** — it must exhaust every box — so a seed of that kind is worth exactly nothing.

It has a different mechanism. Every box costs a probe, an efficiency test and sometimes a repair, all spent to find *a vector*; a vector already known needs none of them. Seeding here is not "start from a better value" but *do not pay to discover what you already know.*

Proposition 2. *Let $S \subseteq E(P_D)$ be a set of efficient points. Placing each $a \in S$ in the box list by locating the unique box containing it and splitting that box around a by* (5)*, then running the enumeration, returns the same F and the same $E(P_D)$, still complete by Proposition 1.*

Proof. The split (5) is disjoint and covers the box minus $\{Z \leqslant Z(a)\}$, so a point whose vector has not been removed lies in exactly one child; the location is therefore well defined, and a seed whose vector has already been removed is dominated by an earlier one and is discarded. The operation performed on the list is exactly the one the enumeration performs when its probe returns a: same centre, same rows, same children. The proof of Proposition 1 appeals only to the centres being feasible and to the region shrinking, both of which hold verbatim. Efficiency of the seeds is what makes the *recorded* vectors non-dominated, and is required for that reason alone. □

So the heuristic decides how much of the front has to be *discovered*, and nothing about what is returned.

6.2 Which heuristic, and why the verification is free

Two sources of efficient points were compared.

The **augmented weighted Tchebychev program** is the natural candidate: its optima are efficient by construction, for any strictly positive weights and any positive augmentation, so its output needs no verification. A **Pareto local search** with an archive proves nothing, so every member has to be put through the exact efficiency test.

That verification looks like the deciding cost and is not, because of an accounting that holds only here: **the enumeration was going to pay the same efficiency test anyway**. Every probe a seed removes was going to be followed by one. The verification cancels, and the probe is the gain. What a seed really costs is its share of the walk.

Table 5: The two sources, per efficient point.

	cost of one point	verification	front covered
Tchebychev scalarisation	20.90 ms	none needed	33–100%
Pareto walk (whole archive)	0.011 s total	one test each	95% median

And a scalarisation is not cheaper than the probe it replaces: one probe costs 9.48 ms and 0 branch & bound nodes, one scalarisation 20.90 ms and 21 nodes, cheaper on only 7 of 40 instances. The probe maximises a linear direction over D plus ordinary rows and usually needs no branching at all; the scalarisation minimises a *maximum*, at the price of a continuous variable and p rows, and it branches.

Consistently with that arithmetic, seeding from the generator *loses*, and loses *more the more of the front it seeds*: with a wide weight grid it covers 100% of the front and runs at 2.61× the unseeded time.

6.3 The order in which seeds are handed over

Placing k seeds was expected to remove k probes. In the median it does (+1.00 over 152 instances), but on 5 of those 152 the seeded run spent *more*: the pre-split imposes a box structure the search would not have chosen, and a box it would have absorbed into a larger one now exists separately and must be probed.

The probe maximises a fixed direction, so the search splits first around whatever scores highest on it. Sorting the seeds by that same direction reproduces the structure the search would have built.

Table 6: Probes against the unseeded enumeration, by the order the seeds arrive. Twenty-seven instances, none censored; the probe count is deterministic, and every arm is checked to return the same front, so no ordering can buy probes by losing vectors. Generated from the campaign's JSON.

order	probes vs unseeded	IQR	raised the count on
archive order	0.83×	[0.74, 0.92]	2 of 27
probe order	**0.69**×	[0.67, 0.75]	**none**
reversed (control)	0.90×	[0.83, 1.05]	9 of 27

The reversed arm is what makes the direction causal rather than a coincidence of this sample. Reversed order is *equally sorted* and differs from probe order only in direction — yet probe order raises the probe count on none of the twenty-seven instances, while reversed raises it on nine, and its median sits with archive order rather than with probe order. What the seeding exploits is therefore the search's own direction, and not the tidiness of a sorted list.

The gap is large enough to decide the hybrid: 0.69× the probes against 0.83×, so roughly two fifths of what the ordering buys is lost by handing the same seeds over the way the archive produces them — which is exactly what a straightforward implementation does.

6.4 The algorithm

Algorithm 1 is the whole method. Phases 1–3 are the heuristic contribution and cost no integer program beyond the verification; phases 4–6 are exact and carry the proof.
Two lines carry the paper. Line 8 is the ordering of Section 6.3: without it the method is *slower* than the pure exact one. And the loop of phase 6 pays no efficiency test, by Theorem 1.

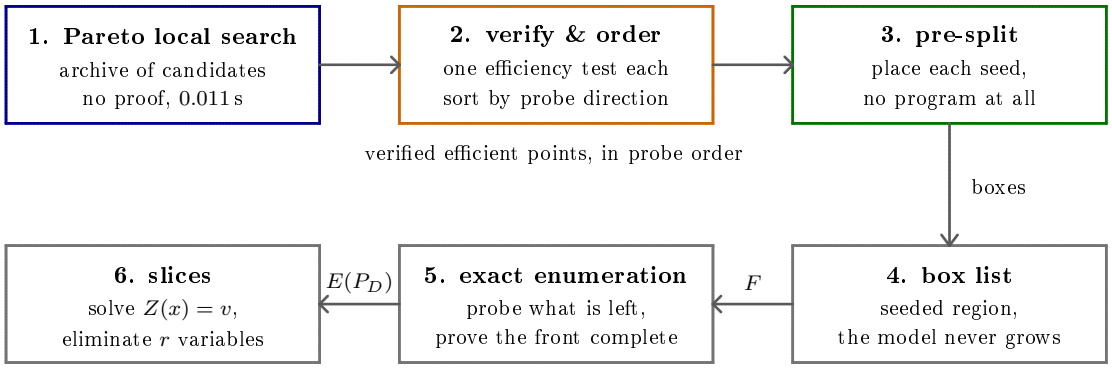


Figure 1: Process structure. Only verified efficient points cross from the heuristic side to the exact side, and they cross in probe order. Nothing crosses back: the heuristic never sees the exact phase's findings, which is why its cost is a one-off.

Algorithm 1 HybridCompleteSet(D, Z, Φ) — all of $E(P_D)$, and the optimum of (P_E)

Phase 1 — explore (heuristic, no guarantee)
$A \leftarrow$ ParetoLocalSearch(D, Z, Φ) ▷ archive of mutually non-dominated points
Phase 2 — verify and order
$S \leftarrow \emptyset$; $V \leftarrow \emptyset$
for all $x \in A$ **do**
  $a \leftarrow x$ **if** x passes the exact efficiency test **else** a dominator of x that does
  if $Z(a) \notin V$ **then** $V \leftarrow V \cup \{Z(a)\}$; $S \leftarrow S \cup \{a\}$
  end if
end for
sort S by $\pi^\top a$ **decreasing**, where $\pi = \sum_k c_k$ is the probe direction
Phase 3 — pre-split (no program at all)
$\mathcal{B} \leftarrow \{$ the root box $\}$; $F \leftarrow \emptyset$
for all $a \in S$ **in that order do**
  $B \leftarrow$ the unique box of $\mathcal{B}$ containing a ▷ none: a is dominated by an earlier seed; skip
  $F \leftarrow F \cup \{Z(a)\}$; $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B\}) \cup$ Split(B, a) ▷ by (5)
end for
Phase 4–5 — exact enumeration of what is left
while $\mathcal{B} \neq \emptyset$ **do**
  pop B from $\mathcal{B}$
  $x \leftarrow \arg\max\{\pi^\top x : x \in D \cap B\}$ ▷ the probe
  if infeasible **then continue** ▷ B is empty: drop it
  end if
  $a \leftarrow x$ **if** efficient **else** a dominator that is
  $F \leftarrow F \cup \{Z(a)\}$; $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B\}) \cup$ Split(B, a)
end while
▷ F is now the complete non-dominated set (Proposition 1, unaffected by the seeding by Proposition 2)
Phase 6 — slices
$E(P_D) \leftarrow \emptyset$
for all $v \in F$ **do**
  solve (6) for r variables; enumerate the remaining $n - r$ ▷ Section 4.3
  $E(P_D) \leftarrow E(P_D) \cup \{\, x \in D : Z(x) = v \,\}$ ▷ all efficient, by Theorem 1
end for
return $E(P_D)$ and $\arg\max_{x \in E(P_D)} \Phi(x)$

7 Two worked examples, step by step

Both are run by the implementation and the figures are emitted from that run; nothing below is drawn by hand. In every figure a filled green square is an efficient point, a grey outline a dominated one, an orange ring a point the walk supplied, red shading the region a split removes, and blue dashed rectangles the boxes currently open.

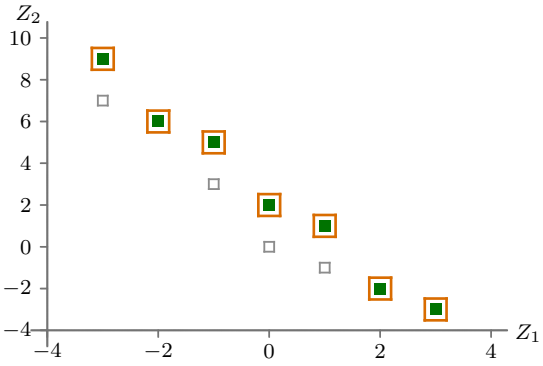
7.1 Two variables

$$D = \{\, x \in \mathbb{Z}^2_+ :\ -2x_1 + x_2 \leqslant 0,\ \ 6x_1 + x_2 \leqslant 21,\ \ -2x_1 + 4x_2 \leqslant 6 \,\},$$

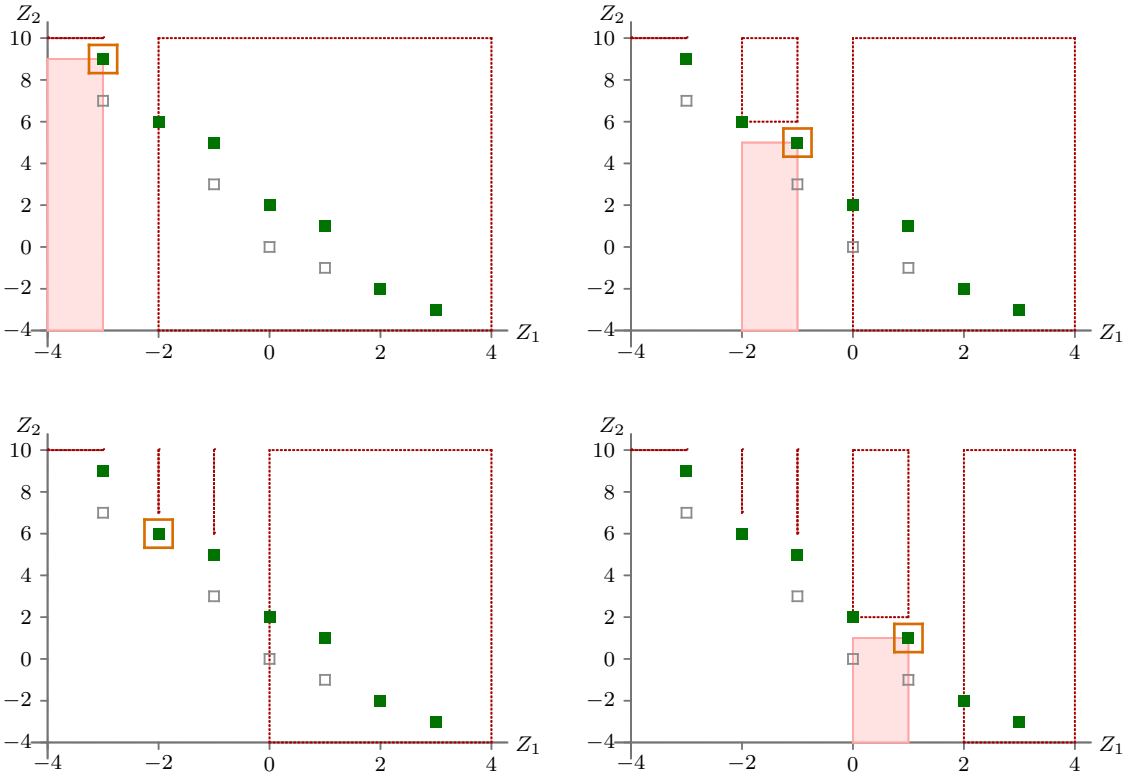
$$Z(x) = (x_1 - 2x_2,\ -x_1 + 4x_2), \qquad \Phi(x) = \frac{x_1 + x_2 - 1}{5x_1 + x_2 - 1}.$$

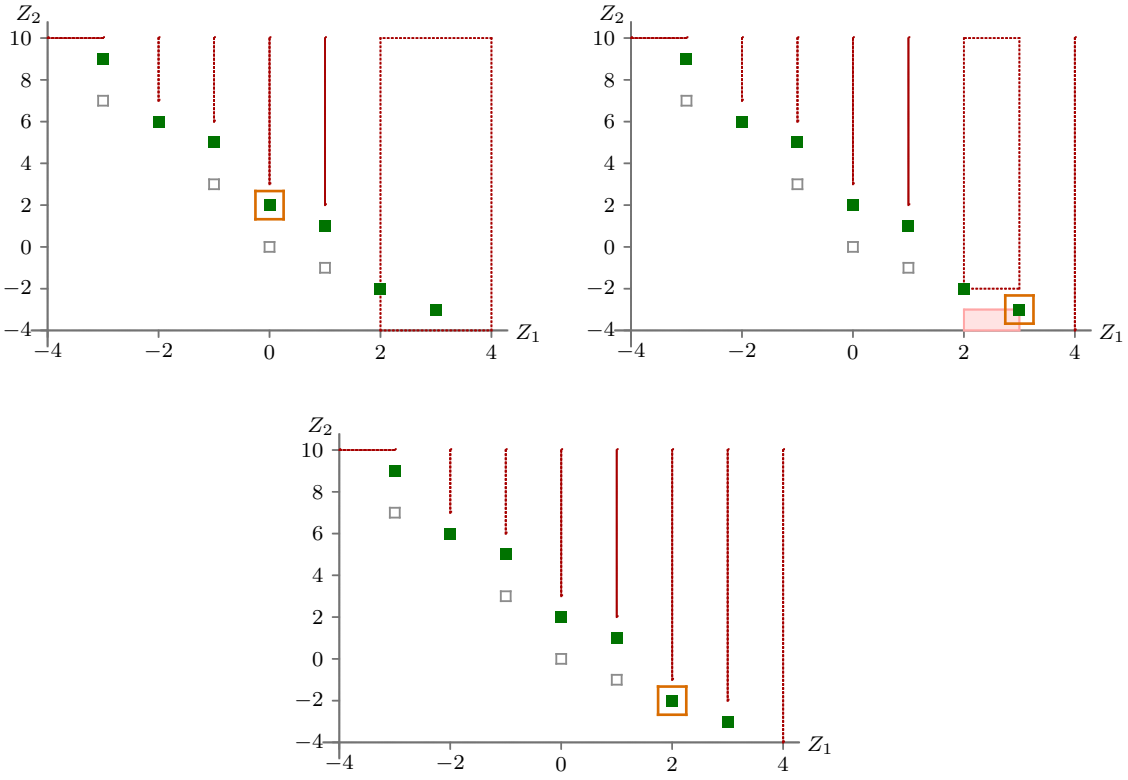
There are 7 non-dominated vectors. The pure enumeration spends **15 probes**; the hybrid spends **8**.

Step 0 — the walk. The Pareto local search returns an archive of 7 points. Each is put through the exact efficiency test, all 7 pass, and their criterion vectors are distinct: the walk has found the *entire* front before a single box exists. It has proved nothing — that is still ahead — but nothing remains to be *discovered*.



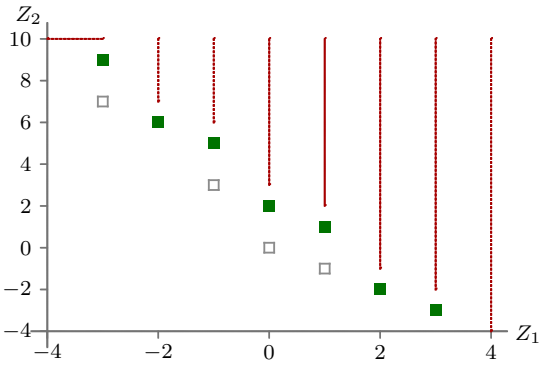
Steps 1–7 — placing the seeds. The seeds are sorted by $\pi^\top x$ decreasing and placed one at a time. Each step locates the unique open box containing the seed, deletes $\{Z \leqslant Z(a)\}$ from it (red), and replaces it by the $p = 2$ children (dashed). **No integer program is solved in any of these steps.**





Watch the open region shrink toward the staircase boundary of the front. After step 7 the list holds 8 boxes, and every one of them lies strictly beyond the front.

Steps 8–15 — the exact phase, which finds nothing. Each of the 8 remaining boxes is probed once. All 8 come back infeasible, and the list empties, which is what *proves* the front complete (Proposition 1).



This is the hybrid in one picture: the heuristic did the *finding*, at 0.011 s and 7 efficiency tests, and the exact method did the *proving*, at 8 probes instead of 15. Neither phase can be dropped — an archive without the box list proves nothing, and the box list without the archive pays to rediscover all 7 vectors.

Step 16 — slices, then (P_E). Each of the 7 vectors is sliced by (6). Here every slice is a single point, so $|E(P_D)| = 7$, and $\max_{E(P_D)} \Phi = 5/17$ at $x = (3, 3)$ — the same answer the decision-space method returns.

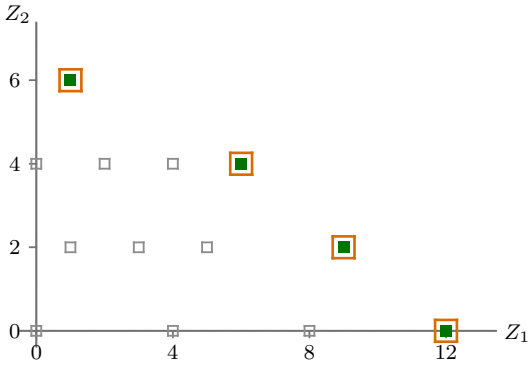
7.2 Three variables

$$D = \{\, x \in \mathbb{Z}^3_+ :\ x_j \leqslant 3,\quad x_1 + x_2 + 2x_3 \leqslant 5,\quad 2x_1 + 3x_2 + x_3 \leqslant 6 \,\},$$

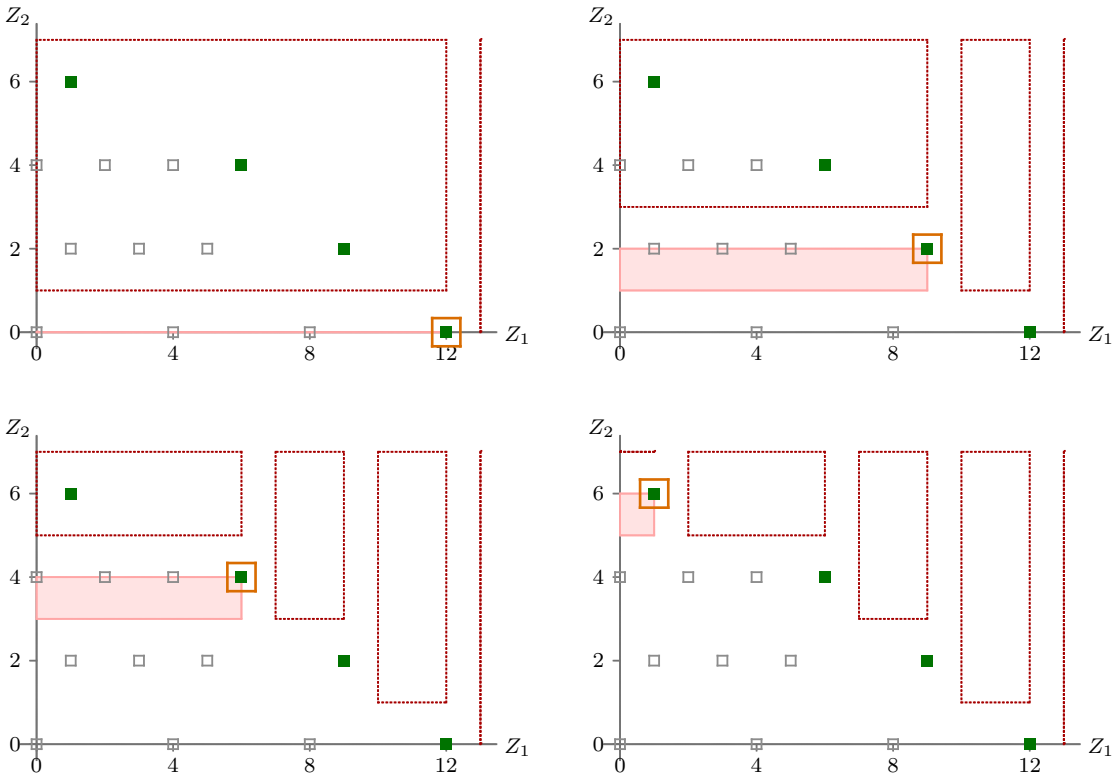
$$Z(x) = (4x_1 - x_2 + x_3,\quad 2x_2 + 2x_3), \qquad \Phi(x) = \frac{-2x_1 + 4x_2 - 3x_3 + 1}{x_1 + 2x_2 + x_3 + 1}.$$

The criterion space is still two-dimensional — $p = 2$ — so the same picture works with $n = 3$, which is the point of showing it: **the method's cost is driven by p and by $|F|$, not by n.** There are 4 non-dominated vectors; the pure enumeration spends 9 probes, the hybrid 5.

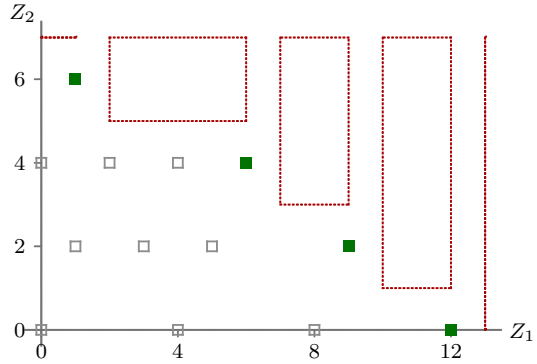
Step 0 — the walk. The archive holds 4 points, all verified efficient, with distinct vectors $(12, 0)$, $(9, 2)$, $(6, 4)$ and $(1, 6)$ — again the whole front.



Steps 1–4 — placing the seeds. In probe order the seeds are $(3, 0, 0)$, $(2, 0, 1)$, $(1, 0, 2)$, $(0, 1, 2)$, which is the front read from the Z_1 end downwards — exactly the order the probe would have met them.



Steps 5–9 — the exact phase. Five boxes remain and all five probe infeasible.



Step 10 — slices. Note x_3 enters Φ but the pairing is still needed: by Remark 1 a vector can carry several points with different Φ, so the maximiser is taken *on the slice* rather than taking whichever point the repair landed on.

7.3 What the two examples show that a table cannot

In both, the walk covers the front completely and the exact phase finds *nothing new*. That is the good case, and it is the common one — the walk covers 95% of the front in the median over the campaign of Section 8. It also shows the ceiling: the exact phase still probes 8 and 5 boxes respectively to *prove* emptiness, and no seed can remove those. The hybrid removes the cost of discovery and leaves the cost of proof untouched, which is why its margin is 1.31× and not 15/8.

8 Computational results

All arithmetic is exact and rational, with no floating point anywhere, so every comparison is between two exactly solved problems. Instances are drawn from a documented pseudo-random family; the statistic is the *median of the per-instance ratios* with the interquartile range, never a ratio of totals, which would be decided by the single heaviest instance.

What the baseline is. The decision-space side of every comparison below is **our own reimplementation** of the published method, not the authors' code, and the instances are ours, not their published benchmarks. Both sides share the same simplex, the same exact rational arithmetic, the same branch-and-bound and the same machine, and the reimplementation follows the printed algorithm step for step, with the two places where that pseudo-code is internally inconsistent corrected and the corrections documented. The comparison therefore *isolates the algorithmic difference*, which is a cleaner question than code against code — that would also measure language, solver backend and engineering effort. The price is that **absolute seconds carry no information outside this implementation**: pure Python over exact rationals is far slower than any production solver, equally on both sides, so only the ratios transfer and only ratios are quoted. A comparison against the authors' own implementations on their own benchmarks has *not* been done and is not claimed; until it is, every margin here reads "this algorithm beats that algorithm when both are implemented the same way", not "this work beats those papers".
The largest front measured here holds 302 vectors. A separate campaign of 150 instances at $n = 6 \ldots 9$ measured the front enumeration at 1.33× and the complete set at 1.31× with the probe count at 0.72× and never worse — the gap between 1.31× and 1.36× is the walk sizing of Section 8.2, which those runs predate. In all of them the answer is identical — same front, same $E(P_D)$, same optimum of (P_E) — which the experiments assert rather than assume.

Table 7: The hybrid against the pure exact enumeration, 120 instances over eight configurations. "Cover" is the share of the front the walk supplies.

configuration	instances	cover	ratio
$n = 6$, $p = 3$	20	100%	1.26×
$n = 7$, $p = 3$	20	100%	1.36×
$n = 8$, $p = 3$	20	100%	**1.59**×
$n = 9$, $p = 3$	20	98%	1.45×
$n = 10$, $p = 3$	12	91%	1.38×
$n = 7$, $p = 4$	20	98%	1.33×
$n = 12$, $p = 3$	6	78%	1.35×
$n = 12$, $p = 5$	2	45%	1.14×
all	**120**		**1.36**×, faster on 116

8.1 Fractional criteria

Everything above uses *linear* criteria, which is a real gap in a paper about ratios: the Tchebychev comparison of Section 6 requires them, since $z_k^* - Z_k(x)$ is not linear when Z_k is a ratio. The Pareto walk has no such requirement, so the hybrid can be measured where the problem statement actually lives.

Table 8: Genuinely fractional criteria: every Z_k a ratio with a strictly positive denominator on D. Ten instances per row.

configuration	$\lvert F\rvert$	cover	ratio
$n = 5$, $p = 2$	7	100%	1.54×
$n = 6$, $p = 2$	2	100%	1.41×
$n = 6$, $p = 3$	22	100%	1.35×
$n = 7$, $p = 3$	36	100%	1.25×
all 40			**1.39**×, [1.21, 1.63], faster on 34

The hybrid pays on the fractional case at least as well as on the linear one, and the walk covers the front completely there. Nothing in the construction distinguishes the two: the seeds are placed by (5), whose rows (3) are integer-valued for ratios as much as for linear criteria.

8.2 How much walking is enough

The walk has one free parameter pair — restarts and neighbour budget — and the hybrid's whole cost advantage is that the walk is cheap next to the search.

Table 9: Sweeping the neighbour budget, 18 instances at $n = 7, 8$.

budget	walk	cover	ratio
250	0.096 s	45%	1.08×
1000	0.080 s	82%	1.31×
4000	0.100 s	100%	**1.36**×
16000	0.100 s	100%	1.35×

Coverage saturates and so does the gain; past saturation the walk terminates on its own, so more budget costs nothing and buys nothing. That reading holds *at this size* — and generalising it is exactly the mistake the next subsection is about.

8.3 Five instance families, and a prediction that was wrong

Everything above comes from one pseudo-random generator, so the magnitudes could belong to it rather than to the method. Four further families move the quantities the mechanism depends on, and move them a long way: `loose` raises $|D|$ by an order of magnitude while leaving $|F|$ alone; `knapsack` uses two capacity rows with all coefficients positive, the shape most of the applied literature has; `conflicting` gives the criteria opposing signs, which makes $|F|$ large; `aligned` correlates them, which makes it small.

Table 10: The hybrid on five structurally different families, 15 instances per cell, 150 in all. Identical front and identical optimum on every one.

family	n	p	$\lvert F\rvert$	cover	ratio	faster on
`mixed`	7	3	8	100%	**1.37×**	13/15
`mixed`	8	4	30	97%	1.30×	15/15
`loose`	7	3	9	83%	1.29×	14/15
`loose`	8	4	44	86%	1.23×	14/15
`knapsack`	7	3	12	100%	1.33×	15/15
`knapsack`	8	4	41	90%	1.27×	14/15
`conflicting`	6	3	107	100%	1.14×	13/15
`conflicting`	7	3	105	93%	1.14×	13/15
`aligned`	7	3	6	100%	1.31×	12/15
`aligned`	8	4	11	84%	1.14×	13/15
all 150					**1.23×**	**136**

Every family pays, between 1.14× and 1.37× in the median, so the advantage is not an artefact of the generator it was found with. That is the result this section was run for.

The prediction it refutes. We expected `conflicting`, with $|F| \approx 107$, to be the family the hybrid liked *most* — a large front is much to discover, and discovery is what a seed removes — and `aligned`, with $|F| = 6$, to be the one it liked least. **The opposite holds**: `conflicting` is the worst of the five at 1.14×, and the margin narrows with $|F|$ *within* every family too: 1.37 → 1.30, 1.29 → 1.23, 1.33 → 1.27, 1.31 → 1.14 as $|F|$ grows from 8 to 30, 9 to 44, 12 to 41, 6 to 11.

Why, and what bounds it. A seed removes *one* probe, and the enumeration spends between 2.3 and 7.5 probes per vector — the rest go to boxes that turn out empty, which no seed can remove. So the *probe-count* saving is capped at one over that figure, by construction, and it shrinks as the ratio grows. Sweeping p at $n = 7$ over 72 instances shows both:

Table 11: Twelve instances per row. The probes-per-vector column is a deterministic count; the time column is not, and the machine's noise floor is roughly ±30% per instance.

p	$\lvert F\rvert$	probes / vector	cover	probes kept	time
2	4	2.3	100%	—	1.38×
3	8	3.0	100%	0.68	1.48×
4	16	4.0	98%	0.75	1.33×
5	30	4.9	99%	0.82	1.21×
6	38	6.4	99%	0.87	1.24×
7	54	7.5	100%	—	1.10×

The probe-count column is monotone and is the part that is certain: 32% of the probes removed at $p = 3$, 13% at $p = 6$. **It does not, however, bound the time.** A seed removes a probe

that lands on a *non-empty* box, which carries an efficiency test behind it, while the probes that remain are mostly on empty boxes and are refuted at once — so the work removed is dearer than the average probe. Measured, 1/(probes kept) and the time ratio agree only loosely, and at $p = 6$ the time ratio exceeds it. We therefore report the probe bound, which is exact, and decline to convert it into a ceiling on time, which the measurement does not support.

What does grow with $|F|$ is the walk's cost: one efficiency test per archive member, so it grows with the front while the saving does not. A large front buys no more and costs more, which is the ordering in Table 10 — and the same mechanism as the $n = 12$, $p = 5$ case of Section 8.4 reached from a different direction.

8.4 Where it fails, why, and the one line that fixes it

At $n = 12$ the hybrid degrades to 1.00×, and at $n = 12$, $p = 5$ with $|F| = 187$ to 0.97×. Our first explanation was that the probe count outgrows $|F|$, so the share a seed can remove shrinks. **The measurement refutes it**: probes per vector are flat across the whole range — 2.4, 2.7, 2.7, 2.8, 2.8, 3.0, 3.9, 5.2 — with no trend.

What collapses is coverage, and the speed-up tracks it line for line:

Table 12: Coverage, the share of probes a seed removes, and the speed-up.

	cover	probes saved	ratio
$n = 6$	87%	29%	
$n = 9$	79%	27%	
$n = 10$	65%	24%	1.19×
$n = 12$, $p = 3$	35%	9%	1.00×
$n = 12$, $p = 5$	12%	2%	0.97×

Nothing in the exact phase degraded. A fixed budget of 8 restarts and 4000 neighbours is a fixed amount of work against a target that grows, and 4000 was chosen at $n = 7, 8$ and generalised.

Widening the walk alone, touching nothing else, confirms the diagnosis and says which knob matters.

Table 13: Widening the walk on the two instances that failed. Restarts are what move coverage; the walk's own time barely changes.

	$n = 12$, $p = 3$		$n = 12$, $p = 5$	
restarts / budget	cover	ratio	cover	ratio
8 / 4000	35%	1.03×	12%	0.98×
16 / 20000	95%	**1.61**×	46%	**1.20**×
32 / 60000	95%	1.53×	93%	1.12×
64 / 150000	95%	1.44×	89%	1.17×

So the fix is one line — the first round is sized to the instance, at $\lceil n/4 \rceil$ times the restarts — and it is what Table 7 already uses. *Not* a second constant: replacing 4000 by 20000 would repeat the error that produced the collapse.

And coverage is not the objective. The $p = 5$ column of Table 13 says something we did not expect and that bounds the whole idea: 46% **coverage gives** 1.20× **and** 93% **gives** 1.12×. Buying the last half of the front costs more walking than the probes it removes.

A walk that restarts in doubling rounds until one finds nothing new makes that concrete. It does exactly what it promises — at $n = 12$, $p = 5$ it takes coverage from 45% to 97% — and it is slower: 1.02× against 1.14× there, and a median 1.32× against 1.36× over the same 120

instances, winning 109 of them against 116. The quantity to maximise is not how much of the front the heuristic finds but the balance between what finding it costs and what the seeds save, and those two part company as p grows.

A correction to our own reading. We first reported the collapse as "$n = 12$, $p = 3$: 35% coverage, 1.00×" from *one* instance. Over six seeds of that configuration the unmodified walk averages 78% coverage and 1.35×. The collapse is real and severe where $|F|$ is large — 45% coverage at $|F| = 302$ — but it is not as general as one instance made it look.

8.5 Two further comparisons

The margin on front enumeration tracks $|F|$, not n. Every vector is a cut, and in decision space a cut costs p binaries and $p+1$ rows, so a larger front means larger sub-problems there and nothing at all in criterion space. Over 45 instances swept across constraint tightness:

Table 14: Criterion space against decision space on complete front enumeration.

$\|F\|$	instances	ratio (median)	max
1–4	26	2.7×	9.8×
5–12	17	7.5×	23.8×
13+	2	31.4×	38.0×

Seed quality explains the choice of heuristic. On 30 paired instances, scoring a seed by the non-negative gap $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$:

Table 15: The two sources as seeds for (P_E) itself.

	median gap	exactly optimal
Pareto archive	0.000	24/30
Tchebychev	0.170	8/30

The ordering is stable across every family tried; the magnitudes are not.

For context, the underlying substitution of criterion space for decision space is worth the following on (P_E) itself, thirty instances per row at $n = 6$:

Table 16: Decision space against criterion space on (P_E).

p	ratio (median)	IQR	sub-programs	b&b nodes
2	1.52×	[1.25, 1.91]	1.72×	1.28×
3	2.69×	[1.87, 3.72]	1.40×	1.79×
4	4.99×	[2.39, 11.62]	1.19×	2.68×
5	7.93×	[3.61, 19.98]	1.17×	3.16×
6	14.57×	[7.90, 25.17]	**1.02**×	**5.37**×

The last row is the cost model of Section 2 read off the measurement: at $p = 6$ the two methods solve *the same number* of sub-programs and one still takes 14.57× as long, so the whole margin lies in the cost of a single sub-problem, whose branch & bound explores 5.37× the nodes because it carries p binaries and $p + 1$ big-M rows per cut.

Measurement discipline. The machine used has a noise floor of roughly ±30% per instance even at per-instance minima over five repeats, so a small effect cannot be resolved in seconds and no number of repeats fixes that. Deterministic work counts — sub-programs, branch & bound nodes, simplex pivots — are reported next to the times and are identical on every run and every machine. Runs stopped by a time budget are reported as censored and excluded from medians rather than counted as their budget.

9 Negative results, and a wall

We report these at the same length as the positive ones, because they delimit where hybridisation can help at all.

9.1 Four changes that remove a quarter of the sub-problems and buy nothing

Table 17: Four attempts on the sub-problem count.

change	sub-problems removed	cost
early exit on a hopeless bound	25%	no measurable time
free contradiction filter	24.8%	0.98× pivots
range filter (ideal, anti-ideal)	+11.3%	1.03× pivots
dominated-corner filter	8–23%	1.077× pivots

All four are exact and one-sided: they discard only boxes that provably hold nothing, so they can change the cost and not the answer. Only the second is kept, and it is worth 2%. After four attempts the reason is a property rather than an accident:

> **In this search, removing sub-problems does not remove time, because the sub-problems that can be removed cheaply are the ones that were already cheap.**

73% of all boxes are empty, and an empty box is refuted by the simplex almost at once: every probe in these families costs *zero* branch & bound nodes, because the relaxation is integral or infeasible at the root. A filter that catches empty boxes is catching the cheapest work in the search. The only change reported here that goes round this wall is Section 4.3, which attacks an *exponent* rather than a count.

9.2 A single-program completeness certificate

After the walk seeds 95% of the front, 93% of the remaining boxes are empty and the enumeration is almost purely a *proof of emptiness*. But "is there a feasible point not dominated by any of the k known ones?" is **one** question, and the box list answers it by splitting it into hundreds. Formulation (4) answers it in one integer program with k cuts, built once and never iterated — which removes exactly the growth that makes that formulation expensive in Section 2.

It is sound: 13 of 30 instances were proved complete by a single program, with no wrong verdict. It is also 4.5× slower in the median and 100× slower at worst (239 s against 1.95 s), because k cuts mean kp binaries in one model — 88 of them at $k = 22$, $p = 4$.

This answers a question wider than an optimisation. Criterion space is the better *search* because the model never grows; the certificate tests whether it is also the better *proof*, where there is no iteration and no growth at all. **It is, by** 4.5×: answering many cheap questions beats answering one question with 88 binaries.

9.3 What the failures have in common

Four hybridisations were attempted in this line of work. Handing the exact search the unfinished work of the decision-space method ties it. Seeding the optimisation search from the Tchebychev program never pays, because that program steers in criterion space relative to the ideal point and never looks at Φ. Seeding the *enumeration* from the same program loses, because a scalarisation costs more than the probe it replaces. Only the Pareto walk pays, and only in probe order. The rule that survives all four:

> A hybridisation pays only where the exact method has a mechanism the first phase can feed **more cheaply than the exact method feeds it itself**.

The first three fail the second half of that test. The fourth passes it because its verification is a test the exact method had already budgeted for.

10 Correctness: what the code checks, and what it merely assumed

Section 8 compares this method against reference methods, so the comparison is only worth as much as the references are. We audited the implementation against that standard and found eleven places where a stated hypothesis was never tested. All eleven are fixed and carry regression tests; the measured results are unchanged. We report them because the pattern behind them is more instructive than any one of them, and because a reader entitled to doubt the numbers in Section 8 is entitled to know how they were defended.

10.1 The pattern

Every flag in the implementation that claimed something had been *proved* was computed from control flow — the loop finished, no sub-problem remained — and not from the hypotheses the proof actually needs. "The box list is empty" was reported as "the front is complete", which is sound exactly when D is bounded and the denominators are positive, and those two facts were assumed rather than established. The same substitution appeared independently in the front enumeration, in both optimisation searches, in the completeness certificate and in the two reference oracles.

The second half of the pattern is sharper. Every parameter added to the API in order to *save work* — a precomputed variable box, a precomputed front, a set of seeds, an incumbent and its value — is a way for a caller to supply a hypothesis without proving it. Each was introduced as an optimisation and each silently became a trust assumption. A caller passing a variable box is asserting that the box contains D; nothing downstream can detect a box that does not, because every routine that receives one searches it honestly and reports a complete-looking answer about a feasible set that is not D.

Both halves were invisible to the test suite for one reason: all 93 tests then in the suite fed *valid* inputs. They established that the method is correct when its hypotheses hold, which was never in question; they could not see that nothing enforced them. The suite now contains 103 tests, and the ten added for the audit all feed deliberately invalid input and require a refusal; three more were added later for the admissible class of Section 5, bringing the suite to 106.

10.2 The findings

Table 18 lists them. Two deserve comment.

Finding 10 was in the completeness certificate of Section 9.2, which searches $\{x \in D : \Phi(x) >$ value$\}$ inside a caller-supplied box. A box smaller than D hides challengers, and the certificate then returns *valid* on a point that is not optimal: with the box $[1, 1]$ it certified an efficient point at $\Phi = 1/9$ while the true optimum is $5/17$. This is the worst location in the package for such

a defect, because the certificate is what validates answers beyond the size at which brute force reaches — a false proof is not a wrong answer but a wrong *check* on other answers.

Finding 11 is the same defect one level further out, in the two reference oracles themselves. Between them they are called from about 45 sites in the test suite and the examples, each passing a box written by hand. Handed a box smaller than D neither failed nor returned less with a warning: they answered confidently about the efficient set of a different feasible set, under the names $E(P_D)$ and "the exact optimum of (P_E)". On $x_1 + x_2 \leqslant 6$, $x_1 - x_2 \leqslant 2$, whose efficient set holds five points and whose optimum is $2/5$, the box $[1, 1]$ yielded one efficient point reported as $E(P_D)$, and $1/3$ reported as the exact optimum. A clipped oracle does not produce a wrong answer, it produces a wrong *standard*, and a method measured against it looks correct precisely when it agrees with the clipping.

Table 18: The eleven findings. "Silent" means the routine returned a confident, complete-looking answer; none of the eleven raised, and none was detectable downstream.

	hypothesis that was not checked	silent consequence
1	D bounded	empty front marked *complete*; both searches reported *infeasible*
2	$D_k(x) > 0$ on D	one vector returned as the whole front while a feasible point dominates it
3	the front handed in belongs to this problem	a foreign front completed into a point set called $E(P_D)$
4	the variable box contains D (slice enumeration)	clipped slices lose efficient points
5	Φ is defined on every slice	the point list and the value list fell out of step
6	a handed-over incumbent is feasible	a search started below an unreachable bound
7	a handed-over incumbent's value is its own Φ	the search pruned against a bound no point realises
8	seeds are efficient	inefficient points entered the box list unprobed
9	the returned point is efficient when state was handed over	an inefficient point returned as the answer to (P_E)
10	the certificate's box contains D	a suboptimal point certified *proved*
11	the reference oracles' box contains D	$E(P_D)$ and $\max \Phi$ redefined; agreement with them became meaningless

10.3 What the audit did not find

The mathematics survived unchanged. We re-derived and re-tested the four propositions the method rests on: the slice characterisation of Section 4, the completeness of the front enumeration, the disjointness of the p-way split, and the validity of the big-M in (4). The last was additionally checked numerically on 220 point pairs, with no violation and with the bound exactly attained, which is what makes it the smallest valid choice rather than merely a safe one. All eleven findings were in the *enforcement* of hypotheses, never in the reasoning that uses them.

Three consequences for the results of Section 8. First, every number there was produced by passing the box the model derives for itself, so none of them was affected; the hybrid still reaches the whole of $E(P_D)$ at $1.36\times$ the pure exact method with all eleven guards active. Second, running the suite with the finding-11 check in place passes 103/103, and the four examples that use the

oracles all run clean, which is the evidence — not the assumption — that none of the roughly 45 hand-written boxes was ever clipping D: the trap sat in the API and never in a shipped measurement. Third, the scaling study still certifies every instance from $n = 10$ to $n = 20$ after finding 10, so the guard that closed it did not invalidate the study that rests on it.

The guards are cheap enough to leave on permanently, which is why they are not a debug mode. Boundedness is one integer program for the whole problem. Positive denominators cost one program per *genuinely fractional* criterion and nothing for linear ones. The variable box is n programs, cached on the problem, so every entry point can afford to re-ask. Verifying a seed is the exact efficiency test the seed needed anyway before it could be trusted. Together they are a fixed cost paid once per problem, invisible against the sub-problem counts of Section 8.

Remark 2. *The honest reading of this section is that a method is not its proof. Each of the eleven was a place where the implementation claimed a proof's conclusion while skipping its premise, and none of them would have been caught by testing that the method gets the right answer on valid input — which is what the suite did for* 93 *tests. They were caught by asking, for each hypothesis stated in a docstring, which line of code establishes it. Eleven times the answer was none.*

11 Conclusion

Keeping the unexplored region in criterion space removes the growth that dominates decision-space methods for (P_E), and the same substitution carries to complete enumeration of the non-dominated set. Theorem 1 then turns that enumeration into the complete efficient set at no efficiency test at all, and solving the slice equations rather than pruning on them makes the remaining enumeration 49× cheaper. On top of that, a Pareto local search whose verified archive is handed over in probe order reaches all of $E(P_D)$ at 1.31× the pure exact method over 150 instances, with identical answers.

The negative results are the boundary of the positive one. A seed is worth what the exact method can do with it, and in an enumeration — which has no bound to feed — it is worth exactly the discovery it replaces, priced against what discovery would have cost. That pricing is also what limits the idea: once the front is large enough that covering it costs more walking than the probes it saves, no amount of heuristic effort helps, and the honest instrument for deciding is the balance rather than the coverage.

Limitations. What a seed can remove is bounded, and Section 8.3 bounds it from measurement: a seed removes one probe, the enumeration spends 2.6 to 6.4 probes per vector, and the rest go to boxes that turn out empty and that no heuristic can remove. That caps the probe saving at 32% at $p = 3$ and 13% at $p = 6$. It does not cap the *time*, since the probes a seed removes are the dearer ones, and we do not claim a time ceiling the measurement does not support. Removing the remaining work would mean attacking the emptiness proof, on which Section 9 reports four failed attempts.

The advantage also depends on the walk staying cheap next to the search, and Section 8.4 shows where that stops: at $|F| = 302$ the walk covers 45% of the front and the margin falls to 1.14×, with more walking making it worse rather than better. We have no rule that says in advance how much walking an instance deserves; the instance-sized first round is a measured heuristic, not a bound.

The instances span five families with $n \leqslant 12$ and $p \leqslant 6$; within that range the direction is stable while the magnitude varies by a factor of two between families, which is itself the honest measure of how far a single number should be carried. The *directions* reported here are explained by mechanisms that are themselves measured — sub-program counts, node counts, pivot counts — but the *magnitudes* are properties of that family and should not be read as expected values elsewhere.

Where this sits in the literature. Two parts of this construction are prior art and are not claimed here. Criterion-space decomposition for optimising over an efficient set is established: Boland, Charkhgard and Savelsbergh (EJOR, 2017) work in the full-dimensional criterion space and decompose it at each iteration, which is structurally Section 3 with a *linear* secondary objective, and their balanced box, boxed line and triangle splitting methods are the bi-objective precursors. Metaheuristic–exact hybridisation likewise has a known taxonomy, in which "run a metaheuristic to approximate the Pareto set, then let an exact method refine it" is exactly the scheme of Section 6.

What the fractional literature does *not* appear to do is work in criterion space. Optimising a linear fractional function over an integer efficient set has a sustained literature — Drici, Ouail and Moulaï (Annals of OR, 2018), a 2015 RAIRO-OR paper, Ouail and Chergui (arXiv:1907.02036), Chaiblaine, Moulaï and Cherfaoui (arXiv:2003.05364), a 2021 Optimization paper on quadratic objectives, and the stochastic and interval-coefficient variants of Younsi-Abbaci and Moulaï (2021–2023) — and all of them search decision space with branch and bound or cutting planes. So what is plausibly unaddressed is narrower than the method: the *combination*, together with what makes the transfer exact rather than approximate — the integer-valued e_k of (3), which replaces an estimated minimal step by $e_k \geqslant 1$ exactly, and the slice results of Section 4 — and what that combination then buys, which is the admissible class of Section 5. That last point is the one we would defend first, because it does not rest on being the fastest at anything: it is a statement about which problems can be posed at all.

This is a statement about what we did and did not find, not a claim of novelty, and two limits are worth naming. No full text in that list has been read; it was assembled from abstracts and listings, so a method an abstract calls "branch and bound" may do something in criterion space the abstract does not mention. And one paper bears directly on the complete-set result of Section 4 and must be read before it is called new: *Generating the efficient set of multi-objective integer linear plus linear fractional programming problems*, Annals of Operations Research **296** (2021), pp. 735–753.

Reproducibility. The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 106 tests. Several check *properties* rather than answers — the split is a partition, the metaheuristic never hands over an inefficient point, a budgeted run never claims completeness — since a method that silently lost efficient points would still agree with the optimum on favourable instances. Per-instance records of every experiment are stored so that each table can be rebuilt and re-checked without re-running it.

References

This list is a starting point and is not yet verified. It was assembled from search results, abstracts and publisher listings; no full text in it has been read, because the environment this work was prepared in could not reach the publishers. Entries whose note says VERIFY have a field that was not confirmed, and fields that could not be found are left out rather than invented — an invented page range is worse than a missing one, since it looks checkable. The two marked CRITICAL bear directly on what this work may claim as new.

References

[1] TODO: author list not confirmed by search. A linear fractional optimization over an integer efficient set. *RAIRO-Operations Research*, 2015. VERIFY – article ro140036, issue 2.

[2] TODO: author list not confirmed by search. Generating the efficient set of multiobjective integer linear plus linear fractional programming problems. *Annals of Operations Research*, 296:735–753, 2021. CRITICAL – VERIFY authors.

[3] TODO: author list not confirmed by search. Quadratic optimization over a discrete Pareto set of a multi-objective linear fractional program. *Optimization*, 70(7), 2021. VERIFY authors and pages.

[4] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. A criterion space search algorithm for biobjective integer programming: The balanced box method. *INFORMS Journal on Computing*. VERIFY year, volume and pages.

[5] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. The triangle splitting method for biobjective mixed integer programming. VERIFY venue, year and pages.

[6] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. A new method for optimizing a linear function over the efficient set of a multiobjective integer program. *European Journal of Operational Research*, 2017. CRITICAL – VERIFY volume and pages.

[7] Yacine Chaiblaine, Mustapha Moulaï, and Youcef Cherfaoui. An exact method for optimizing two linear fractional functions over the efficient set of a multiobjective integer linear fractional program. arXiv:2003.05364, 2020. VERIFY author list. The two-decision-function problem, solved in decision space with branch and bound plus cuts.

[8] Wassila Drici, Fatma Zohra Ouail, and Mustapha Moulaï. Optimizing a linear fractional function over the integer efficient set. *Annals of Operations Research*, 2018. VERIFY author list, volume and pages.

[9] J. G. Ecker and I. A. Kouada. Finding efficient points for linear multiple objective programs. *Mathematical Programming*. VERIFY everything – not confirmed by search.

[10] Banu Lokman and Murat Köksalan. Finding all nondominated points of multi-objective integer programs. *Journal of Global Optimization*, 2013. VERIFY volume and pages.

[11] Fatma Zohra Ouail and Mohamed El-Amine Chergui. Hyperbolic optimization over the integer efficient set of MOILFP. arXiv:1907.02036, 2019. VERIFY author list.

[12] Melih Özlen and Meral Azizoğlu. Multi-objective integer programming: A general approach for generating all non-dominated solutions. *European Journal of Operational Research*, 2009. VERIFY volume and pages.

[13] John Sylva and Alejandro Crema. A method for finding the set of non-dominated vectors for multiple objective integer linear programs. *European Journal of Operational Research*. VERIFY everything – not confirmed by search.

[14] Leila Younsi-Abbaci. Optimizing a linear fractional function over an integer efficient set. *Reliability: Theory & Applications*, 2025. The paper this implementation follows. VERIFY: the repository README records "No 4 (40), Vol. 11, March 2025", whose volume and year do not obviously agree – check the issue.